

Code Review Summary

For our project, we did manual code review and AI-assisted code analysis. In the manual code review we looked for the code smells listed in the task description, as well as anything else we found “bad”. We did this by simply following the logic of the code, trying to be as critical as possible. For the AI-assisted code analysis, we used GitHub Copilot which has a full overview of the codebase. Therefore we could simply ask it to look through all project files and identify code smells.

Code Smell 1

Name

Low cohesion

Description

The service *complianceService* both handles business logic (finding violations and evaluating log compliance) and also contains a method *listPolicyRules* performing filtered reads from the database. The filtered read is used both internally in the service, and by the *policyController*. Reads like these could instead be put in a repository, which both the service and the controller could use. This way, the service would only contain business logic, which improves cohesion.

Method

Manual code review: Analysing the concerns of the files.

Code snippets

```
> export async function listPolicyRules({ courseId, assignmentId }) { ...  
  }  
|  
> export async function evaluateUsageLogCompliance(usageLogInput) { ...  
  }
```

Both the “query function” and the business logic function *evaluateUsageLogCompliance* are in the service.

Code Smell 2

Name

Duplicated validation logic

Description

In *logController*, validation is done for the usage log payload. However, the logic is duplicated in two places. If validation were to change, it would have to be changed in both places. Duplication like this also bloats the controller, especially if more endpoint are added that also needs the same validation.

Method

Manual code review: Looking for duplicate code.

Code snippets

```

39     const usagePayload = resolveUsagePayload(req.body);
40     if (!userId) {
41         throw new AppError(
42             "Unauthenticated request. Provide x-user-id.",
43             401,
44         );
45     }
46
47     const validation = validateUsageLogPayload(usagePayload);
48     if (!validation.valid) {
49         throw new AppError(validation.message, 400);
50     }

```

```

116     const usagePayload = resolveUsagePayload(req.body);
117     const validation = validateUsageLogPayload(usagePayload);
118     if (!validation.valid) {
119         throw new AppError(validation.message, 400);
120     }

```

The usagePayload is validated in the exact same way for both endpoints.

Code Smell 3

Name

Duplication/inconsistency

Description

In *logController* there is created a helper function to resolve the user id in requests. It is used by one of the endpoints. However, the other endpoint does this resolving without using the helper function. Meaning the logic is duplicated. If the resolving of the user is changed later on, then there is a danger that it is not changed everywhere.

Method

Manual code review: Looking for duplicate code.

Code snippets

```

function resolveUserId(req) {
    const body = req.body || {};
    const nestedPayload =
        body && typeof body.payload === "object" && body.payload !== null
        ? body.payload
        : null;

    return (
        req.headers["x-user-id"] ||
        body.userId ||
        nestedPayload?.userId ||
        req.query.userId
    );
}

```

The function to resolve the user id.

```
export async function deleteLogsByUser(req, res, next) {
  try {
    const requestingUserId = req.headers["x-user-id"];
    const targetUserId = req.params.userId;

    if (!targetUserId) {
      throw new AppError("userId is required", 400);
    }
  }
}
```

The endpoint that resolves it by itself.

Code Smell 4

Name

Dead code

Description

The arguments *req* and *next* of the function *errorHandler* are never used.

Method

Manual code review

Code snippets

```
export function errorHandler(err, req, res, next) {
  const statusCode = err.statusCode || 500;
  const response = {
    error: {
      message: err.message || "Internal server error",
    },
  };

  if (err.details) {
    response.error.details = err.details;
  }

  if (process.env.NODE_ENV !== "production" && err.stack) {
    response.error.stack = err.stack;
  }

  res.status(statusCode).json(response);
}
```

GitHub Copilot Prompt:

"can you perform a code review of the code base? this is what you should look after:

You will apply code review practices to identify and reason about code smells—structural weaknesses that can reduce maintainability, readability, reliability, or performance.

Examples of code smells include (but are not limited to): long or overly complex components, duplicated logic, primitive obsession, large classes, long parameter lists, temporary fields, shotgun surgery, dead or unreachable code, excessive comments, deeply nested JSX, mixing UI and business logic, and missing error handling.

for every code smell you identify please list them in this format:

- (1) Name of the smell*
- (2) Short description*
- (3) Identify method*
- (4) Relevant original code snippets*

if its easier please give me the list as its own document"

GitHub Copilot Response:

1) Duplicated Validation Logic Across Layers

(1) Name of the smell

Duplicated logic

(2) Short description

Required-field rules are implemented in multiple places (backend validator, frontend draft checker, frontend submit validity), which creates drift risk when requirements change.

(3) Identify method

Cross-file contract comparison of validation rules in backend and frontend.

(4) Relevant original code snippets

```
// backend/src/utils/validators.js
const requiredFields = [
  "courseId",
  "assignmentId",
  "toolName",
  "assistanceType",
  "contributionDescription",
];
```

```
// frontend/src/features/usage-compliance/hooks/useComplianceCheck.js
const requiredKeys = [
  "courseId",
```

```

    "assignmentId",
    "toolName",
    "assistanceType",
    "contributionDescription",
  ];

  // frontend/src/features/usage-compliance/components/UsageLogForm.jsx
  const isValid = useMemo(() => {
    return (
      formState.userId.trim() &&
      formState.courseId.trim() &&
      formState.assignmentId.trim() &&
      formState.toolName.trim() &&
      formState.contributionDescription.trim()
    );
  }, [formState]);

```

AI suggestion is *accepted* as it correctly identifies duplicated logic which could impact maintainability.

2) Primitive Obsession / Magic Strings for Domain Concepts

(1) Name of the smell

Primitive obsession

(2) Short description

Core domain concepts (*assistanceType*, *ruleType*, IDs) are represented as raw strings in many places without a shared contract. This increases typo risk and weakens refactorability.

(3) Identify method

Inspection for repeated string literals representing domain types.

(4) Relevant original code snippets

```

// backend/src/models/UsageLog.js
assistanceType: {
  type: String,
  required: true,
  enum: [
    "brainstorming",
    "editing",
    "translation",
    "code-review",
    "research",
  ],
}

```

```

    "summarization",
    "generation",
    "other",
  ],
},

// frontend/src/features/usage-compliance/types/index.js
export const assistanceTypeOptions = [
  "brainstorming",
  "editing",
  "translation",
  "code-review",
  "research",
  "summarization",
  "generation",
  "other",
];

// backend/src/services/complianceService.js
const evaluators = {
  assistance_type_not_allowed: ...,
  required_field_non_empty: ...,
  max_ai_contribution: ...,
  keyword_block: ...,
};

```

AI suggestion was *accepted* because it correctly identifies primitive obsession.

3) Long / UI-Heavy Component (Maintainability Risk)

(1) Name of the smell

Long component / overly complex component

(2) Short description

UsageLogForm combines state handling, validation, data mapping, and a large JSX tree in one file, making it harder to maintain and test.

(3) Identify method

Component-level complexity review (responsibility count + repeated JSX sections + form mapping logic).

(4) Relevant original code snippets

```

// frontend/src/features/usage-compliance/components/UsageLogForm.jsx
export function UsageLogForm({ ... }) {
  const [formState, setFormState] = useState(defaultUsageLogDraft);

```

```

const isValid = useMemo(() => { ... }, [formState]);

function updateField(fieldName, value) { ... }
async function handleSubmit(event) { ... }

return (
  <form className="card" onSubmit={handleSubmit}>
    ... many grouped inputs/labels/selects/textareas ...
  </form>
);
}

```

AI suggestion was *accepted* because the specific component has low cohesion (multiple responsibilities).

4) Duplicate Payload Construction (Shotgun-Surgery Trigger)

(1) Name of the smell

Duplicated logic / shotgun surgery risk

(2) Short description

Payload mapping is repeated in both `updateField` and `handleSubmit`. Any payload shape change requires editing multiple blocks.

(3) Identify method

Local duplication scan inside the same component.

(4) Relevant original code snippets

```

// frontend/src/features/usage-compliance/components/UsageLogForm.jsx
onDraftChange({
  courseId: nextState.courseId,
  assignmentId: nextState.assignmentId,
  toolName: nextState.toolName,
  assistanceType: nextState.assistanceType,
  contributionDescription: nextState.contributionDescription,
  aiContributionPercent: Number(nextState.aiContributionPercent || 0),
  promptExcerpt: nextState.promptExcerpt,
});

```

```

// frontend/src/features/usage-compliance/components/UsageLogForm.jsx
const payload = {
  courseId: formState.courseId,
  assignmentId: formState.assignmentId,
  toolName: formState.toolName,

```

```
assistanceType: formState.assistanceType,  
contributionDescription: formState.contributionDescription,  
aiContributionPercent: Number(formState.aiContributionPercent || 0),  
promptExcerpt: formState.promptExcerpt,  
};
```

AI suggestion was *accepted* because it correctly identifies code duplication and the potential of shotgun surgery.

5) Missing Per-Item Validation in Batch Compliance Endpoint

(1) Name of the smell

Missing error handling / missing validation

(2) Short description

The batch endpoint validates only that **logs** is an array, but not each log structure. Invalid entries can silently degrade compliance quality.

(3) Identify method

API input validation review for endpoint robustness.

(4) Relevant original code snippets

```
// backend/src/controllers/logController.js  
if (!courseId || !assignmentId || !Array.isArray(logs)) {  
  throw new AppError(  
    "courseId, assignmentId and logs[] are required",  
    400,  
  );  
}
```

```
const compliance = await evaluateSubmissionCompliance({  
  courseId,  
  assignmentId,  
  logs,  
});
```

AI suggestion was *accepted* because it could be a robustness issue although not a standard code smell.

6) Exception-Driven Control Flow with Swallowed Cause

(1) Name of the smell

Missing error handling details / broad catch

(2) Short description

Rule evaluation catches all exceptions and emits a generic violation without preserving diagnostics. This masks root causes and complicates debugging.

(3) Identify method

Service-level error-path inspection.

(4) Relevant original code snippets

```
// backend/src/services/complianceService.js
for (const rule of rules) {
  try {
    const evaluator = evaluators[rule.ruleType];
    ...
  } catch (error) {
    violations.push(malformedRuleViolation(rule));
  }
}
```

AI suggestion is *accepted* because it is correctly identifying the broad catch.

7) Potential Dead/Unused Code Path

(1) Name of the smell

Dead or unreachable code (currently unused function)

(2) Short description

`refreshLogs` is exposed by `useUsageLogs` but has no call sites in the current frontend feature, increasing API surface without active usage.

(3) Identify method

Workspace symbol usage search for `refreshLogs`.

(4) Relevant original code snippets

```
// frontend/src/features/usage-compliance/hooks/useUsageLogs.js
const refreshLogs = useCallback(async ({ userId, courseId, assignmentId }) => {
  ...
}, []);

return {
  logs,
  isLoading,
  error,
```

```
refreshLogs,  
submitUsageLog,  
};
```

AI suggestion is *accepted* because the part is unused in the current code.

8) Duplicated Policy Query Rules in Multiple Layers

(1) Name of the smell

Duplicated logic

(2) Short description

Policy filtering logic (`assignmentId` exact-or-null behavior) appears in both controller and service, which can diverge over time.

(3) Identify method

Cross-layer query construction comparison.

(4) Relevant original code snippets

```
// backend/src/controllers/policyController.js  
if (req.query.assignmentId) {  
  query.$or = [  
    { assignmentId: req.query.assignmentId },  
    { assignmentId: null },  
  ];  
}
```

```
// backend/src/services/complianceService.js  
if (assignmentId) {  
  query.$or = [{ assignmentId }, { assignmentId: null }];  
}
```

AI suggestion is *accepted* because it correctly identifies duplicated logic.

9) Sequential Async Processing in Batch Check (Performance Smell)

(1) Name of the smell

Potential performance smell (unnecessary sequential async)

(2) Short description

Batch compliance evaluates each log sequentially with `await` in a loop. For larger batches, this can increase latency significantly.

(3) Identify method

Async flow review in batch-processing service.

(4) Relevant original code snippets

```
// backend/src/services/complianceService.js
const results = [];
for (const logItem of logs) {
  const compliance = await evaluateUsageLogCompliance({
    ...logItem,
    courseId,
    assignmentId,
  });
  results.push({ log: logItem, compliance });
}
```

AI suggestion is *accepted* as it correctly identifies the use of `await` in a `for` loop which can potentially affect performance and data flow.

10) Configured Rule Field Not Used in Response Construction

(1) Name of the smell

Dead data / ignored configuration

(2) Short description

`PolicyRule` requires `educationalMessage`, but violation rendering uses a generic global message and does not include per-rule educational guidance, reducing value of configured data.

(3) Identify method

Model-to-response trace review.

(4) Relevant original code snippets

```
// backend/src/models/PolicyRule.js
educationalMessage: {
  type: String,
  required: true,
  trim: true,
  maxLength: 1200,
},
```

```
// backend/src/services/complianceService.js
function toViolation(rule, trigger) {
  return {
```

```
ruleId: rule.ruleId,  
ruleName: rule.name,  
explanation: `${rule.description}${trigger ? ` Triggered by: ${trigger}.` : ""}`,  
suggestedAction: rule.suggestedAction,  
severity: rule.severity || "medium",  
};  
}
```

```
// backend/src/services/complianceService.js
```

```
const educationalMessage =
```

```
  violations.length > 0
```

```
    ? "Great effort documenting your AI usage. Keep strengthening your independent thinking  
    by applying the feedback below."
```

```
    : "Great job. Your AI usage appears aligned with current course policy.";
```

AI suggestion is *accepted* as it correctly identifies dead data which could impact maintainability.